

SIMATS ENGINEERING
ASSESSMENT TOOL 2- COMPARATIVE ANALYSIS

Course Code:	CSA1202	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL2- COMPARATIVE ANALYSIS
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	Athisaya U	Reg.No.:	192571001
Department:	B.Tech Computer Science and Biosciences	Date:	25/08/2026

ANNEXURE A
COMPARATIVE ANALYSIS

- 1.Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.
- 2.Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.
- 3.Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.
- 4.Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.
- 5.Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

Code of Conduct:

I Athisaya U / 192571001 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Athisaya U

ANNEXURE A

COMPARATIVE ANALYSIS

CO4 | Comparative Analysis

Total Marks: 25

1. Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.

ANSWER

Why the Cache Is Split Into Levels

A single cache would force one impossible compromise. Making it fast means keeping it small, since access time in an SRAM array grows with the number of rows that have to be decoded and the length of the wires that carry the result. Making it large enough to hold a realistic working set means accepting an access time that no pipeline can absorb on every load. The usual solution is to place three caches in series, each one larger and slower than the one above it.

This arrangement works because memory references are not random. **Temporal locality** means a location just referenced is likely to be referenced again shortly; **spatial locality** means neighbouring locations are likely to follow. A small store therefore captures a large share of all accesses, and each level below it only handles what the level above could not hold.

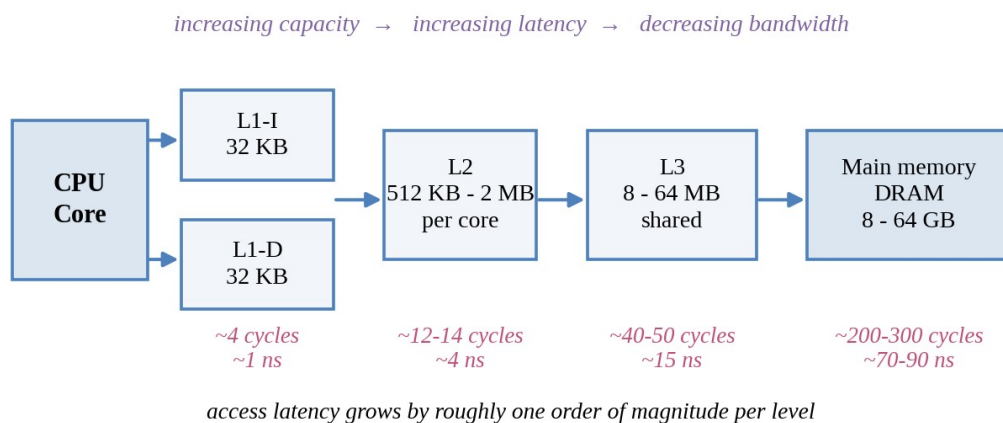


Figure 1: Organisation of the three cache levels between the core and main memory

Comparison of L1, L2 and L3

Parameter	L1 cache	L2 cache	L3 cache
Typical capacity	32-64 KB per core, split into L1-I and L1-D	512 KB - 2 MB per core	8-64 MB shared by all cores
Access latency	~4 cycles (~1 ns at 4 GHz)	~12-14 cycles (~3-4 ns)	~40-50 cycles (~12-15 ns)
Peak bandwidth	~200-400 GB/s per core (two loads + one store per cycle)	~100-200 GB/s per core	~50-100 GB/s shared across cores
Associativity	8-way, kept low to protect latency	8-16 way	12-20 way
Placement	Private, inside the core pipeline	Private per core, adjacent to the core	Shared, on the ring or mesh interconnect
Typical hit rate	~90-95%	~50-80% of L1 misses	~40-60% of L2 misses
Access ports	Multi-ported, several accesses per cycle	Usually single-ported	Banked and shared
Main design goal	Minimum latency	Balance of latency and capacity	Capacity and inter-core data sharing
Cost per bit	Highest	Moderate	Lowest of the three

The same trade-off appears in almost every row of the table. Associativity is a clear example. L1 is limited to eight ways because the tag comparison lies on the critical path, so widening it would slow down every hit. L3 can use twenty ways because its latency is already dominated by the interconnect crossing. In each case the parameter is chosen to suit the role that level performs.

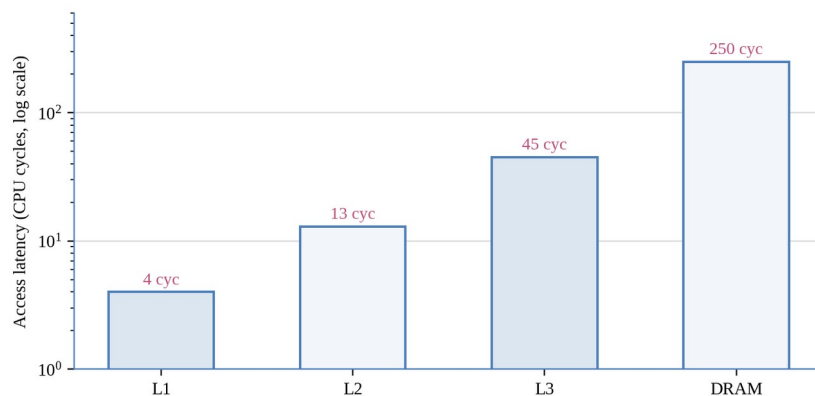


Figure 2: Access latency of each level on a logarithmic scale

Analysis of the Performance Impact

Latency, bandwidth and throughput affect the processor at different times. Latency is exposed when an instruction depends on the value being fetched and the pipeline has nothing else to issue. Bandwidth matters when many independent accesses are outstanding at once and the limit becomes how fast data can be moved rather than how long any one request takes. Throughput is the result the program experiences. Average memory access time links them:

```

AMAT = HitTime(L1)
      + MissRate(L1) x [ HitTime(L2)
      + MissRate(L2) x ( HitTime(L3)
      + MissRate(L3) x DRAMLatency ) ]

```

Worked example (typical values):

```

HitTime(L1) = 4, MissRate(L1) = 0.05
HitTime(L2) = 13, MissRate(L2) = 0.30
HitTime(L3) = 45, MissRate(L3) = 0.50
DRAMLatency = 250 cycles

```

```

AMAT = 4 + 0.05 x [13 + 0.30 x (45 + 0.50 x 250)]
      = 4 + 0.05 x [13 + 0.30 x 170]
      = 4 + 0.05 x 64 = 7.2 cycles

```

A DRAM access costs 250 cycles, but the average access works out to **7.2 cycles**. This is the result of the filtering effect: L1 answers 95% of requests, and most of the small fraction that escapes is caught before it reaches memory. The 250-cycle penalty still applies, but it is incurred rarely enough to have little effect on the average.

The contribution of L3 can be tested by removing it from the same calculation. Without L3 the average becomes $4 + 0.05 \times (13 + 0.30 \times 250) = 8.4$ cycles, a **17% increase**. This is a significant amount for a level that is itself relatively slow, and it justifies keeping a third level even though every L2 miss must pass through it.

- **L1 protects the pipeline.** Every load passes through it, so its hit time lies directly on the critical path. Increasing the size of L1 to reduce misses would slow down the 95% of accesses that were already hitting, so its capacity is deliberately kept small.
- **L2 provides cheap filtering.** At roughly 13 cycles it costs about twenty times less than reaching DRAM. Even a modest hit rate at this level removes a large number of stall cycles from the pipeline.
- **L3 supplies capacity and a sharing point.** Being shared, it also lets cores exchange data without a memory round trip, which becomes important in multithreaded code and in the coherence traffic between cores.
- **Bandwidth becomes the limiting factor under load.** Out-of-order execution and hardware prefetching keep several misses in flight simultaneously. At that point the machine is bandwidth-limited rather than latency-limited, and the wide multi-ported L1 is what sustains the instruction rate.

Application and Suitability

Level	Workload it benefits most	Reason
L1	Tight loops, pointer chasing, register-poor code	Repeated access to a very small working set with dependent loads
L2	Medium working sets such as image tiles or matrix blocks	Holds a block that no longer fits L1 but is reused many times
L3	Multithreaded servers, databases, shared data structures	Large capacity and a single shared copy visible to every core

Conclusion

None of the three levels can be judged on its own, because each is designed around the misses of the level above it. The calculated outcome given earlier makes the point: three levels reduce a 250-cycle memory access to an effective 7.2 cycles, and L3, despite its high latency, accounts for 17% of that improvement. Where a workload has a small, hot working set, L1 and L2 determine its performance almost entirely. Once the working set grows, or is shared between threads, L3 capacity becomes the limiting factor and the hierarchy's benefit shifts downward with it.

The cycle counts and hit rates above are **representative of current desktop and server cores** rather than measurements of one specific part. Exact values shift with clock frequency, process node and cache configuration, so they are quoted as ranges where a range is more honest than a single number.

Result: Three cache levels reduce an average memory access from 250 cycles to 7.2 cycles, and removing L3 alone costs about 17% of that gain. Latency governs dependent loads, bandwidth governs behaviour under parallel misses, and the hierarchy exists because capacity and speed cannot be optimised in the same array.

2. Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.

ANSWER

How the Two Cell Designs Differ

Most of the performance differences between these two memories can be traced back to a single design decision at the level of the bit cell. **SRAM** holds a bit in a pair of cross-coupled inverters, six transistors in total, forming a latch that maintains its own state for as long as power is supplied. **DRAM** holds a bit as charge on a small capacitor reached through one transistor. That charge dissipates, so the cell must be refreshed periodically, and the act of reading it drains the capacitor and forces a write-back.

The difference between six transistors and one, and between a self-sustaining latch and a leaking capacitor, accounts for the density gap, the latency gap, the refresh requirement and the difference in cost per bit discussed below.

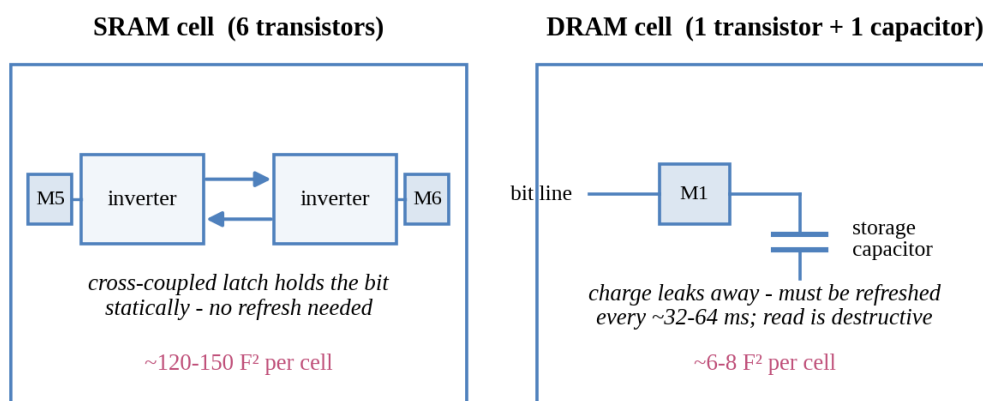


Figure 3: Cell structure of SRAM and DRAM

Comparison of SRAM and DRAM

Parameter	SRAM	DRAM
Cell structure	6 transistors forming a bistable latch	1 transistor + 1 capacitor
Cell area	~120-150 F ² per bit	~6-8 F ² per bit, roughly 15-20x denser
Access latency	~0.5-15 ns depending on array size	~70-90 ns from the CPU, ~14-18 ns internal row access
Refresh	None required while powered	Every ~32-64 ms, consuming bandwidth and energy
Read behaviour	Non-destructive	Destructive — the row must be written back
Bandwidth	~200-400 GB/s per core on-chip, multi-ported	~50-100 GB/s per DDR5 channel-pair, shared
Interface	On-die, wide and short wires	Off-chip bus with controller queuing and bus turnaround
Standby power	Higher static leakage per bit	Lower per bit, but refresh adds continuous activity
Cost per bit	High	Low — roughly an order of magnitude cheaper
Typical use	Registers, L1/L2/L3 caches, tag arrays	Main memory, frame buffers, large working sets

Cell area is the most significant row in the table. A 15-to-20-fold density advantage is the reason a system can afford gigabytes of DRAM but only megabytes of SRAM. The other entries in the DRAM column, such as the refresh requirement, the destructive read and the off-chip interface, are costs accepted in exchange for that density.

Latency, Bandwidth and Throughput

Latency. An SRAM read is fast because the latch is already driving the bit line with a full logic level, so the sense amplifier has almost no work to do. DRAM must first activate a row, wait while the capacitor's small charge disturbs a precharged bit line, amplify that disturbance, and then restore what the read destroyed. Those stages appear individually as tRCD, tCAS and tRP in the timing specification, and their sum is why DRAM latency is measured in tens of nanoseconds where SRAM is measured in fractions of one.

Bandwidth. The difference is smaller here. On-die SRAM benefits from short, wide, multi-ported connections, allowing a single L1 to supply hundreds of gigabytes per second to one core. DRAM cannot match that on any individual access, but it recovers much of the ground through **banking, burst transfer and row-buffer reuse** — once a row is open, successive columns stream out of it rapidly. This mechanism is why DRAM can provide reasonable bandwidth even though its latency remains poor.

Throughput. What the processor actually achieves depends on which resource runs out first. SRAM sustains high throughput for small working sets but cannot be scaled up: a 32 MB SRAM main memory would be prohibitive in both area and cost. DRAM sustains acceptable throughput for large working sets provided the access pattern keeps rows open. **Under random access there is no row-buffer reuse, and throughput falls back towards the raw latency figure** — which is precisely the case the cache hierarchy exists to hide.

Application and Suitability

Design position	Technology chosen	Justification
L1 / L2 / L3 cache	SRAM	Latency is on the critical path; capacity needed is small enough for the area cost to be acceptable
Main memory	DRAM	Capacity per rupee and per square millimetre dominates; latency is hidden by the cache hierarchy
Register file, TLB, tag arrays	SRAM	Extremely small, accessed every cycle, must be multi-ported
Embedded systems with tiny footprints	SRAM only	Removes the refresh controller and simplifies the design

Conclusion

SRAM is the logical choice for cache memory, where access speed matters more than density or cost and the required capacity is small enough for the area penalty to be tolerable. DRAM offers the opposite balance, which makes it the only practical option for main memory once capacity is measured in gigabytes. Neither could replace the other. An all-SRAM machine would be too expensive to build, and an all-DRAM machine would stall on almost every load. The memory hierarchy is the arrangement that allows a system to obtain the latency of the first together with the capacity of the second.

Cell areas are quoted in F^2 , the squared minimum feature size, because this keeps the density comparison meaningful across process nodes. Timing figures are **typical values rather than a specification for any one module**.

Result: The six-transistor latch buys SRAM roughly two orders of magnitude in latency; the one-transistor cell buys DRAM 15-20 times the density and an order of magnitude in cost per bit. Cache memory takes the first trade, main memory takes the second, and the hierarchy exists to make the combination behave like both.

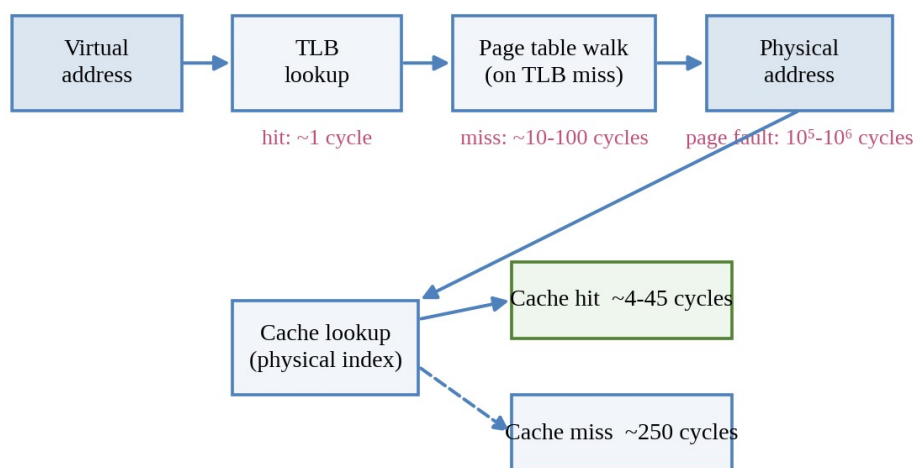
3. Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.

ANSWER

Two Mechanisms That Are Often Confused

Caching and paging look similar in outline. Both move data automatically between a small fast store and a large slow one, both depend on locality of reference, and both are invisible to the program using them. This similarity can be misleading, so one point should be settled before the comparison: they are not alternative solutions to the same problem.

Caching exists to reduce the latency of reaching main memory, and it is implemented entirely in hardware. **Virtual memory with paging** exists to give each process a large, private, contiguous address space and to allow total memory demand to exceed physical RAM. It is implemented jointly, by the MMU and TLB in hardware and by the operating system in software. On a physically indexed cache the two are not even parallel paths — **the address is translated first and the cache is consulted afterwards**, so a single load may involve both.



translation and caching are sequential steps on one access path, not alternatives

Figure 4: Address translation and cache lookup on one access path

Comparison of Paging and Caching

Parameter	Cache memory	Virtual memory with paging
Purpose	Reduce effective access latency to main memory	Provide a large private address space and protection
Levels involved	SRAM cache against DRAM	DRAM against SSD or disk backing store
Managed by	Hardware only, transparent to the OS	Hardware MMU plus operating-system policy
Block size	Cache line, 64 bytes typically	Page, 4 KB typically (2 MB / 1 GB huge pages)
Hit latency	~4-45 cycles	~1 cycle on a TLB hit
Miss penalty	~250 cycles to DRAM	~10-100 cycles for a page walk; 10 ⁵ -10 ⁶ cycles for a page fault
Miss handling	Hardware fetches the line; pipeline stalls	OS traps, schedules I/O and switches to another process
Typical miss rate	L1 ~5%; L3 misses a few per thousand	TLB miss ~1%; page faults rare in a well-sized system

Parameter	Cache memory	Virtual memory with paging
Replacement policy	Approximate LRU in hardware	Clock, LRU approximation or working-set policy in software
Effect on throughput	Removes stall cycles from one thread	Allows many processes to share limited RAM

One row in this table explains most of the others. The miss penalties differ by four to five orders of magnitude, and once that is taken into account, the differences in block size, replacement policy and management responsibility follow directly rather than appearing to be arbitrary design choices.

Consequences of the Miss Penalty Difference

A cache miss costs a few hundred cycles and the pipeline simply waits. A page fault requires an actual I/O operation: on an NVMe SSD at roughly 50 μ s that is around 200,000 cycles at 4 GHz, and on a magnetic disk it runs to tens of millions. Working through the consequences:

- **Block size.** A page is 64 times larger than a cache line. When the dominant cost is setting up the transfer rather than performing it, moving far more data per event is the rational response.
- **Placement policy.** Caches limit associativity to keep hit time short. Pages are **fully associative** — any virtual page may occupy any physical frame — because a fault is far too expensive for a conflict miss to be acceptable, whatever the cost of the comparison hardware.
- **Who handles the miss.** A cache miss is too brief to justify saving and restoring processor state, so hardware resolves it. A page fault lasts long enough that the operating system can profitably schedule a different process meanwhile, converting an otherwise wasted stall into useful work.
- **Write policy.** Caches commonly choose write-back with a dirty bit. Paging has no realistic alternative to write-back, since committing a whole page to disk on every store would be unworkable.

Effect on System Throughput

The two contribute to throughput at different scales. Caching raises **single-thread throughput** by eliminating stall cycles, and the size of that benefit is bounded by the miss rate. Paging raises **system-level throughput** by allowing the combined working sets of many processes to exceed physical memory, keeping more processes resident and the CPU more likely to have runnable work available.

This second benefit has a failure mode that caching does not share. It holds only while the working sets genuinely fit; beyond that point the system enters **thrashing, spending more time servicing page faults than executing instructions**, and throughput collapses rather than degrading gracefully. A cache miss, by contrast, is always bounded by DRAM latency, so no access pattern can cause a comparable collapse.

The TLB is the component where both mechanisms meet, and it is itself a cache — a small associative store of recent translations. Without it, every access would require a page-table walk and virtual memory would be too slow to use. A hit adds effectively nothing; a miss adds a walk of one to four memory references, a cost modern designs partly absorb by caching page-table entries in the ordinary data cache.

Application and Suitability

Requirement	Mechanism that addresses it	Why
Hiding DRAM latency from the pipeline	Cache memory	Operates at cycle granularity in hardware
Running more processes than RAM allows	Paging	Backing store extends apparent memory capacity
Isolation and protection between processes	Paging	Per-process page tables with permission bits
Sharing read-only code between processes	Paging	Several page tables map one physical frame
Sustaining high instruction throughput	Both together	Caches remove stalls; the TLB keeps translation free

Conclusion

Because the two mechanisms address different problems, the useful judgment concerns where each belongs rather than which performs better. Caching is the only viable approach to reducing latency within memory itself, because a few hundred cycles is far too short an interval for software to intervene. Paging is the correct mechanism for capacity, isolation and multiprogramming, because a millisecond-scale event is long enough for a policy decision to be worth making in software. A well-configured system uses both, and sizes physical memory so that page faults remain rare enough never to dominate the throughput that caching was added to protect.

The page-fault cost of 10^5 - 10^6 cycles is derived from the SSD and disk service times given in the table at a 4 GHz clock, rather than quoted directly, so it scales with whichever backing store a particular system uses.

Result: Caching and paging share a principle but differ by four to five orders of magnitude in miss penalty, and that single figure explains their differing block sizes, associativity and division between hardware and software. Caching raises single-thread throughput; paging raises system throughput until working sets outgrow RAM, at which point thrashing removes the benefit entirely.

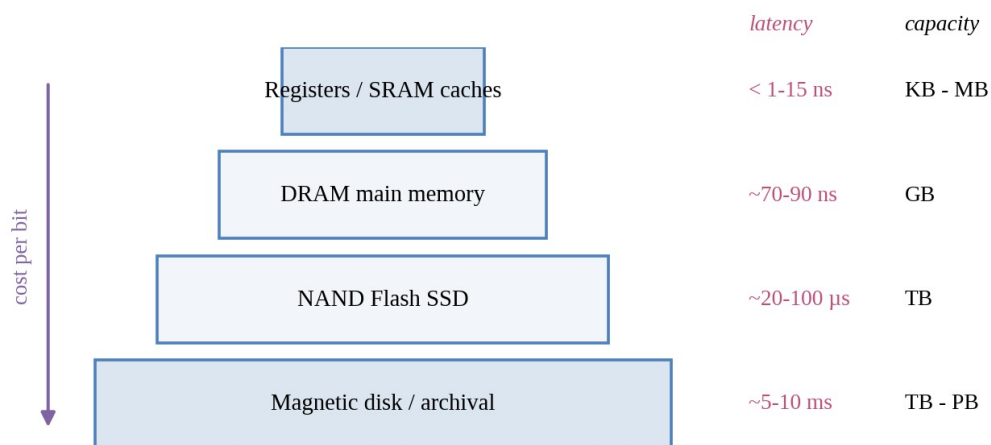
4. Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.

ANSWER

Two Adjacent but Very Different Levels

DRAM and NAND Flash occupy neighbouring positions in the memory hierarchy, yet the step between them is the largest discontinuity anywhere in that structure. DRAM is volatile, byte-addressable working memory. NAND Flash is non-volatile storage that traps charge on a floating gate or in a charge-trap layer and holds it for years without power.

Latency is the difference usually quoted first, but it is not the one that determines where NAND can be used. NAND is organised in pages and blocks rather than bytes, and a block must be erased before any part of it can be rewritten. That organisational constraint, more than the slower cell, is what rules NAND out of certain roles entirely.



each step down trades roughly 10^3 in latency for 10^3 in capacity per rupee

Figure 5: Position of DRAM and NAND Flash in the memory hierarchy

Comparison of NAND Flash and DRAM

Parameter	DRAM	NAND Flash
Volatility	Volatile — contents lost on power removal	Non-volatile — retains data for years unpowered
Access granularity	Byte addressable via the memory bus	Page for read/write (4-16 KB); block for erase (MB scale)
Read latency	~70-90 ns	~20-100 μs, roughly 1000x slower
Write latency	~70-90 ns, symmetric with read	~200-800 μs program; block erase in milliseconds
Bandwidth	~50-100 GB/s per DDR5 channel-pair	~3-14 GB/s per NVMe device across many channels
In-place update	Yes	No — a block must be erased before rewriting
Endurance	Effectively unlimited	~1,000-100,000 program/erase cycles by cell type
Density and cost	Moderate density, higher cost per bit	Very high density (3D stacking), far lower cost per bit

Parameter	DRAM	NAND Flash
Idle power	Continuous refresh power	Near zero when idle
Typical capacity	8-64 GB per system	512 GB - several TB per device

Comparing the latency and bandwidth rows against each other is more revealing than reading either alone. NAND trails DRAM by a factor of about a thousand on a single access but by only about ten on sustained transfer. A device that is three orders of magnitude slower per request, but only one order of magnitude slower on bulk transfer, is clearly suited to some access patterns and unsuitable for others.

Interpreting the Performance Gap

Latency. Two effects combine here. At device level, sensing trapped charge through a series string of NAND cells is intrinsically slower than sensing a DRAM capacitor across a short bit line. At system level, an SSD access must also traverse a controller, a flash translation layer and a PCIe link, each adding fixed overhead that no improvement in the cell can remove. At 50 μ s, one NAND access is roughly 200,000 CPU cycles, which places it well outside the critical path of instruction execution.

Bandwidth. NAND performs far better on this measure because an SSD interleaves requests across many dies and channels operating in parallel. Sequential bandwidth of several gigabytes per second is routine. The gap between the two figures is the clearest practical illustration of why latency and bandwidth must be treated as separate quantities: **NAND can move data in bulk, but it cannot answer a single question quickly.**

Write asymmetry and endurance. NAND writes are slower than reads, cannot be applied in place, and consume a finite endurance budget. The flash translation layer therefore performs wear levelling and garbage collection, introducing **write amplification** and making worst-case latency far less predictable than DRAM's. DRAM exhibits none of these behaviours — reads and writes are symmetric, in place, and effectively unlimited — and this predictability matters as much as the raw speed when a device is being considered for working memory.

Position in the hierarchy. DRAM sits immediately below the caches as main memory because it is byte-addressable and quick enough for demand paging to function. NAND sits below DRAM as backing store and file storage, where its granularity is no longer a problem because a page fault has already committed the system to an I/O operation. The size of the gap between them is exactly what the emerging persistent-memory tiers are attempting to close.

Application and Suitability

Role in the hierarchy	Technology	Reason
Main memory / working set	DRAM	Byte addressable, symmetric access, latency tolerable behind caches
Swap and backing store for paging	NAND SSD	Non-volatile and large; page-fault cost already assumes I/O
File and database storage	NAND SSD	High sequential bandwidth and very low cost per bit
Buffer cache in front of storage	DRAM	Absorbs repeated reads so NAND latency is rarely exposed
Write-back cache inside the SSD	DRAM	Coalesces writes and reduces write amplification

Conclusion

NAND Flash cannot serve as working memory, and the reason is not simply that it is slow. Byte-level updates are impossible without a block erase, worst-case latency is unpredictable because of garbage collection, and endurance is finite. DRAM has none of these constraints, which is why it remains the only realistic choice for main memory. What NAND offers instead — persistence, very high density and a cost per bit far below DRAM — is exactly what a capacity tier requires. The two are

complementary, though not symmetrically. DRAM is used to hide NAND's latency, while NAND supplies the capacity that could not be provided in DRAM at an acceptable cost.

Endurance varies widely with cell type, which is why it is given as a range: SLC sits at the upper end and QLC at the lower. NAND latency and bandwidth are quoted at device level, including controller and interface overhead, since that is what a system actually experiences.

Result: NAND trails DRAM by roughly 1000x in latency but only about 10x in sustained bandwidth, and it cannot update in place. Those two facts together disqualify it as working memory and recommend it as the capacity tier, which is precisely the position it occupies below DRAM.

5. Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

ANSWER

Where MRAM and RRAM Fit in the Hierarchy

The previous comparison identified a gap in the hierarchy: nothing currently occupies the space between DRAM at around 70 ns and NAND at tens of microseconds. MRAM and RRAM are the two leading candidates for that gap. Both are **emerging non-volatile memories** that store a bit as resistance rather than as charge, and because the state is a physical property of the device, both retain data without power and neither requires refresh.

The mechanisms are unrelated. **MRAM**, in its spin-transfer-torque form, uses a magnetic tunnel junction whose resistance depends on whether two magnetic layers are aligned or opposed. **RRAM** uses a metal-oxide layer in which an applied voltage grows or ruptures a conductive filament. Their different physics produces the differences in speed, density and energy that follow.

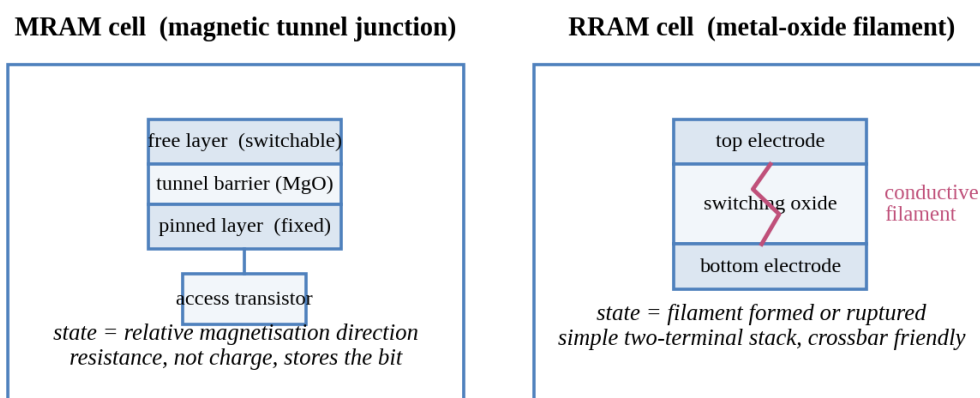


Figure 6: Cell structure of an MRAM magnetic tunnel junction and an RRAM filamentary cell

Comparison of MRAM and RRAM

Parameter	MRAM (STT-MRAM)	RRAM (ReRAM)
Storage mechanism	Magnetisation direction of a free layer in an MTJ	Formation or rupture of a conductive filament in an oxide
Read access time	~2-10 ns, close to SRAM	~10-50 ns, between SRAM and DRAM
Write access time	~5-20 ns	~10-100 ns, generally slower and more variable
Endurance	~10 ¹² -10 ¹⁵ cycles, effectively unlimited	~10 ⁶ -10 ⁹ cycles, far better than NAND but finite
Write energy	Higher — a spin-polarised current must switch the layer	Lower per bit, but forming and reset add variability
Read energy	Low; the read margin (TMR ratio) is modest	Low; the on/off resistance ratio is large and easier to sense
Cell size and density	~20-40 F ² , limited by the switching current	~4-12 F ² , crossbar and 3D stacking friendly
Retention	~10 years, traded against switching current	~10 years, degrades with cycling
Variability	Low, magnetic switching is comparatively uniform	Higher — filament formation is stochastic

Parameter	MRAM (STT-MRAM)	RRAM (ReRAM)
Maturity	In volume production as embedded flash replacement	Mainly embedded and research/neuromorphic use

Two rows account for most of the difference between these technologies, and they point in opposite directions. MRAM is clearly better on variability and endurance, while RRAM is clearly better on cell size. Neither advantage offsets the other, which is why both are still under development instead of one having replaced the other.

Access Time, Throughput and Energy

Access time. MRAM is faster, but the more consequential advantage is that it is more *predictable*. Magnetic switching is a comparatively uniform physical process, so read and write times cluster closely around their mean. Filament formation in RRAM is stochastic, which produces an acceptable average with a long slow tail. Since memory timing must be specified for the worst case rather than the average, **a long tail is far more damaging to a design than a modest increase in mean latency**. This is the main reason MRAM reached volume production first.

Throughput. RRAM's simple two-terminal stack suits crossbar arrays and 3D integration, so a given area holds several times more bits and more of them can be read in parallel. Aggregate throughput can therefore favour RRAM even where each individual access is slower — the same distinction between latency and bandwidth seen with NAND in the previous answer. MRAM needs an access transistor capable of supplying the switching current, which limits its density but gives it higher throughput per access.

Energy efficiency. Three components need separating here. **Static energy** is close to zero for both, and this is their shared advantage over SRAM and DRAM: no leakage-dominated retention and no refresh, so idle energy falls sharply in systems that spend most of their time inactive. **Read energy** is low for both. **Write energy** is where the two differ. MRAM's switching current is substantial and is the principal obstacle to scaling it further down. RRAM writes more cheaply per bit but pays for it in variability and reduced endurance. For a write-heavy workload the comparison can reverse. MRAM's higher per-write energy may still be preferable, because RRAM cells would need over-provisioning and active wear management to survive the same duty cycle.

Application and Suitability

Application	Better suited	Reason
Embedded non-volatile memory in microcontrollers	MRAM	Endurance and predictable timing; already replacing embedded flash
Last-level cache or persistent SRAM replacement	MRAM	Read latency near SRAM with zero standby leakage
High-density storage-class memory	RRAM	Smaller cell, crossbar and 3D stacking give cost per bit
Neuromorphic and in-memory computing	RRAM	Analogue multi-level resistance suits synaptic weight storage
IoT nodes with long sleep intervals	Either, MRAM preferred	Zero retention energy; MRAM adds endurance for frequent logging

Conclusion

The choice between them follows from what a design is limited by. Where speed, endurance and timing predictability matter — embedded non-volatile memory, or roles adjacent to the cache — MRAM is the stronger option, and it is also the more mature technology at present. Where density and cost per bit dominate, RRAM is preferable, and its analogue multi-level behaviour gives it a distinctive advantage in in-memory and neuromorphic computing that MRAM does not share.

Neither displaces DRAM as main memory yet. MRAM is not dense or cheap enough, and RRAM is not yet consistent enough for a system to rely on its worst-case timing. Their realistic near-term role remains the **storage-class memory tier between DRAM and NAND** identified at the start of this

answer, where non-volatility and an access time of tens of nanoseconds are worth more than matching DRAM outright.

Reported figures for emerging memories vary considerably more between devices than those for DRAM or SRAM, since both technologies are still developing and results depend heavily on the material stack and process used. The ranges above should be read as **indicating how the two technologies compare with each other**, not as fixed specifications for either.

Result: MRAM leads on access time, endurance and timing predictability; RRAM leads on density, cost per bit and analogue in-memory computing. Both eliminate refresh and standby leakage entirely, which is their common advantage over DRAM and SRAM, and both are best positioned as a storage-class tier between DRAM and NAND rather than as a replacement for either.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand